Project 3 Report

Numerical Methods and Data Visualization

Student Name: Pouria Morajab
Student ID: 40113026

July 6, 2026

Abstract

This report presents the implementation and results of Project 3. The project includes polynomial interpolation using the Lagrange method, natural cubic spline interpolation, reading execution-time data from an Excel file, drawing bar, line, and box plot charts, calculating the mean execution time of Algorithm 2, and approximating a numerical integral using the trapezoidal rule, Simpson's rule, and Gaussian quadrature.

1 Interpolation

The given data points are:

$$(1,1),\ (2,3),\ (3,5),\ (4,8),\ (5,5),\ (6,2)$$

Two interpolation methods were implemented: Lagrange interpolation and natural cubic spline interpolation. Lagrange interpolation produces a single polynomial that passes through all data points, while the natural cubic spline constructs a separate cubic polynomial on each interval.

1.1 Lagrange Polynomial

The Lagrange polynomial obtained from the program is:

$$P(x) = \frac{7}{40}x^5 - \frac{71}{24}x^4 + \frac{147}{8}x^3 - \frac{1249}{24}x^2 + \frac{1369}{20}x - 31$$

1.2 Natural Cubic Spline

The natural cubic spline pieces are:

$$
\begin{aligned}
1 \le x \le 2: &\quad S(x) = -\frac{39}{209}x^3 + \frac{117}{209}x^2 + \frac{340}{209}x - 1 \\
2 \le x \le 3: &\quad S(x) = \frac{195}{209}x^3 - \frac{117}{19}x^2 + \frac{3148}{209}x - \frac{2081}{209} \\
3 \le x \le 4: &\quad S(x) = -\frac{28}{11}x^3 + \frac{5256}{209}x^2 - \frac{16481}{209}x + \frac{17548}{209} \\
4 \le x \le 5: &\quad S(x) = \frac{470}{209}x^3 - \frac{6768}{209}x^2 + \frac{31615}{209}x - \frac{46580}{209} \\
5 \le x \le 6: &\quad S(x) = -\frac{94}{209}x^3 + \frac{1692}{209}x^2 - \frac{10685}{209}x + \frac{23920}{209}
\end{aligned}
$$

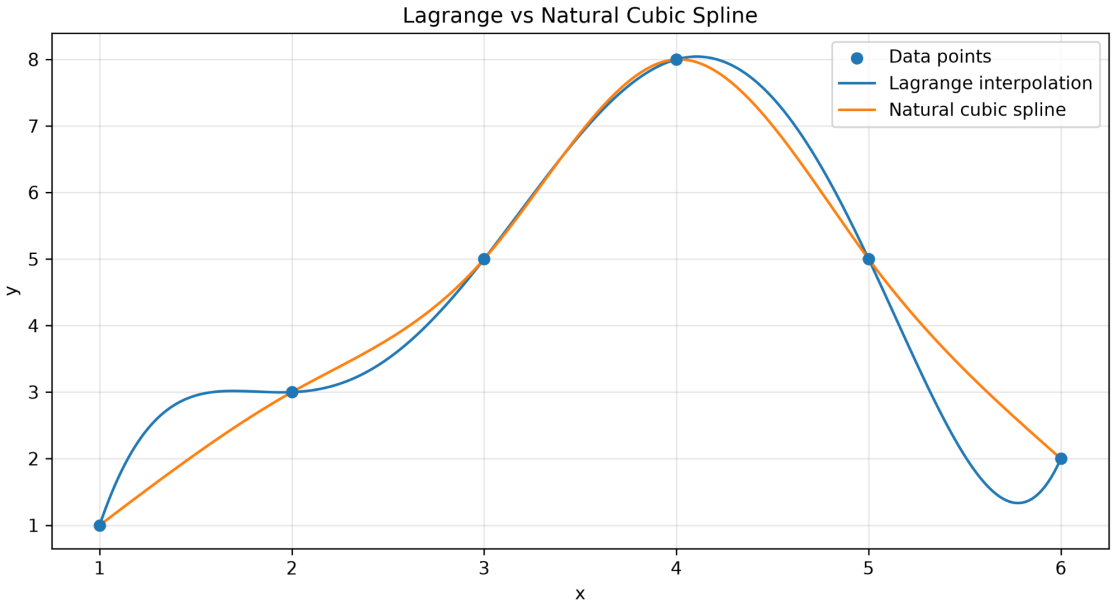

Figure 1: Comparison of Lagrange interpolation and natural cubic spline interpolation.

1.3 Core Code

```
x_points = [1, 2, 3, 4, 5, 6]
y_points = [1, 3, 5, 8, 5, 2]

# Lagrange polynomial
P = 0
for i in range(len(x_points)):
    L = 1
    for j in range(len(x_points)):
        if i != j:
            L *= (x - x_points[j]) / (x_points[i] - x_points[j])
    P += y_points[i] * L

# Natural cubic spline
spline = CubicSpline(x_points, y_points, bc_type="natural")
```

2 Excel Reading and Charts

The execution-time data were read directly from the Excel file. The data used in the current run are shown in Table 1.

Table 1: Execution-time data read from Excel.

Data Size	Alg.1	Alg.2	Alg.3
100KB	50	200	100
200KB	55	220	200
300KB	60	240	300
400KB	65	260	400
500KB	70	280	500
600KB	75	300	600

The program generates three charts from the Excel data: a bar chart, a line chart, and a box plot.

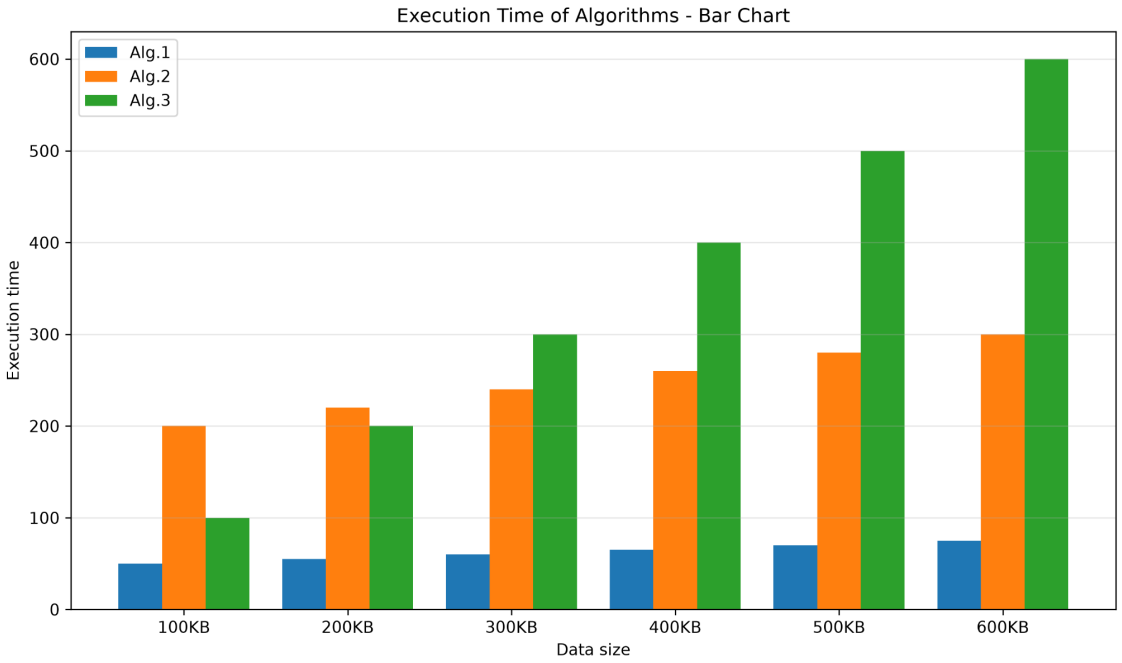

Figure 2: Bar chart of execution times for the three algorithms.

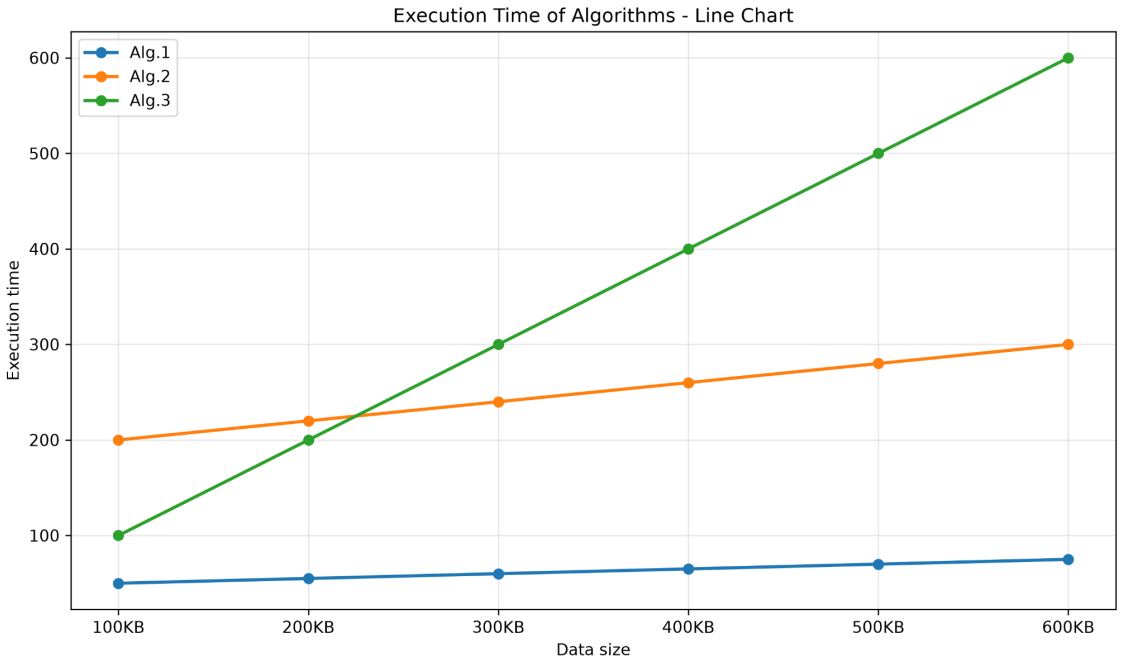

Figure 3: Line chart of execution times for the three algorithms.

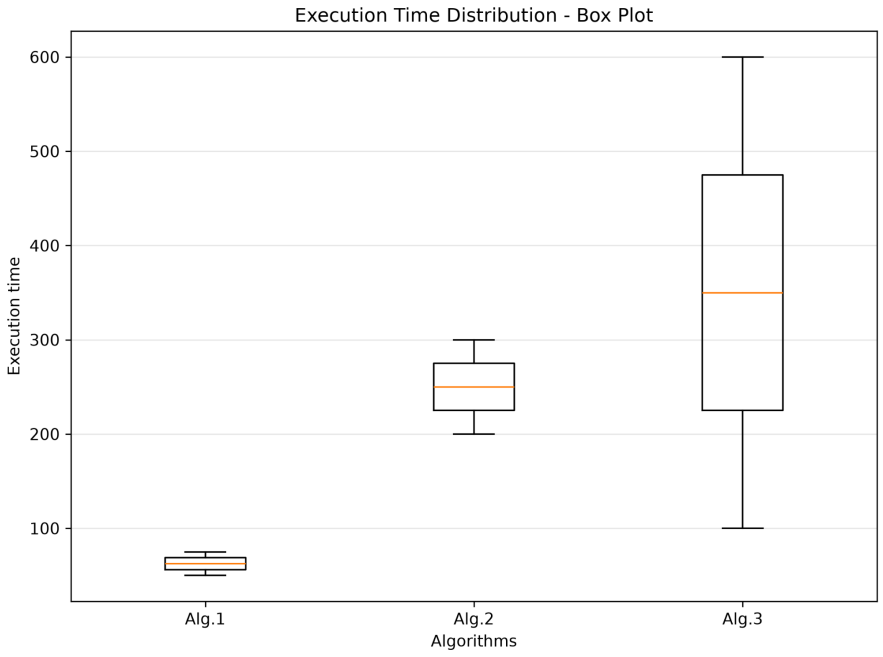

Figure 4: Box plot showing the distribution of execution times.

2.1 Chart Analysis

Algorithm 1 has the lowest execution time and grows slowly as the data size increases. Algorithm 2 has a medium execution time and shows a steady increasing trend. Algorithm 3 has the highest growth rate and becomes much slower for larger input sizes.

2.2 Mean Execution Time of Algorithm 2

The mean execution time of Algorithm 2 for data sizes from 100KB to 600KB is:

$$\frac{200 + 220 + 240 + 260 + 280 + 300}{6} = 250.0$$

3 Numerical Integration

The integral evaluated in this section is:

$$I = \int_0^1 e^{x^2}\, dx$$

The integral was approximated by three numerical methods: the trapezoidal rule, Simpson's rule, and Gaussian quadrature.

Table 2: Numerical integration results.

Method	Approximation
Trapezoidal rule, $n = 1000$	1.462652198954
Simpson's rule, $n = 1000$	1.462651745907
Gaussian quadrature, 5 points	1.462651668019
Gaussian quadrature, 10 points	1.462651745907
Reference value	1.462651745907

Table 3: Absolute errors compared with the reference value.

Method	Absolute Error
Trapezoidal rule	$4.530468948882 \times 10^{-7}$
Simpson's rule	$3.008704396734 \times 10^{-13}$
Gaussian quadrature, 5 points	$7.788850031609 \times 10^{-8}$
Gaussian quadrature, 10 points	$1.332267629550 \times 10^{-15}$

3.1 Comparison

The trapezoidal rule is simple, but it has the largest error among the tested methods. Simpson's rule is much more accurate for this smooth function. Gaussian quadrature with 10 points gives the most accurate result and almost matches the reference value.

3.2 Core Code

```
def f(x):
    return np.exp(x**2)

trap_result = trapezoidal_rule(f, 0, 1, 1000)
simpson_result = simpson_rule(f, 0, 1, 1000)
gauss_5 = gaussian_quadrature(f, 0, 1, 5)
gauss_10 = gaussian_quadrature(f, 0, 1, 10)
```

4 Conclusion

In this project, interpolation methods, Excel-based data visualization, and numerical integration methods were implemented in Python. The Lagrange polynomial and natural cubic spline both pass through the given data points, but the spline provides a smoother piecewise representation. The Excel charts show that Algorithm 1 is the fastest, Algorithm 2 has moderate growth, and Algorithm 3 grows the fastest. For the numerical integral of e^{x^2} over $[0, 1]$, Gaussian quadrature with 10 points produced the most accurate result.

5 GitHub Repository

The project files are available in the following GitHub repository:

`https://github.com/pooriyamm83/Mathemathical_software`